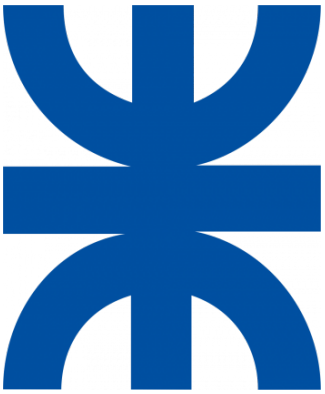
Universidad Tecnológica Nacional de Córdoba

Facultad Regional de Córdoba



PLAYZONE

“Plataforma de venta de juegos digitales”

- ☐ **COMISIÓN:**
  - ➔ 1w3
- ☐ **MATERIA:**
  - ➔ Base de Datos I
- ☐ **ALUMNOS:**
  - ➔ 430253 Fullana Enzo
  - ➔ 421654 Gaiardo Gauna, Giorgio
  - ➔ 421409 López Repoma, Kevin Andrei Jesús
  - ➔ 431713 Martinez Jalil, Rodrigo Elias
  - ➔ 109917 Minelli, Santiago
  - ➔ 430197 Ruiz, Gerónimo
- ☐ **PROFESOR:**
  - ➔ Canovas, Damian Cesar
- ☐ **FECHA DE ENTREGA:**
  - ➔ 10/09/2026

ÍNDICE

1.-ENUNCIADO: **3**
2.-DIAGRAMA E-R: 4
3.-CONSULTAS 5

1.-ENUNCIADO:

La plataforma PlayZone desea contar con un sistema de base de datos que permita administrar y consultar información relacionada con la venta y distribución de videojuegos digitales, así como también la información de sus usuarios, bibliotecas de juegos, compras y actividades realizadas dentro de la plataforma.

La plataforma cuenta con un catálogo de videojuegos pertenecientes a diferentes géneros y desarrollados por distintos estudios. Cada videojuego posee información propia como título, descripción, fecha de lanzamiento, precio, clasificación, desarrollador y/o empresa desarrolladora, además de poder pertenecer a uno o varios géneros.

Los usuarios de PlayZone pueden registrarse en la plataforma proporcionando sus datos personales. Cada usuario posee una biblioteca en la cual se registran los videojuegos que ha adquirido o agregado a su cuenta. Un mismo videojuego puede formar parte de las bibliotecas de una gran cantidad de usuarios.

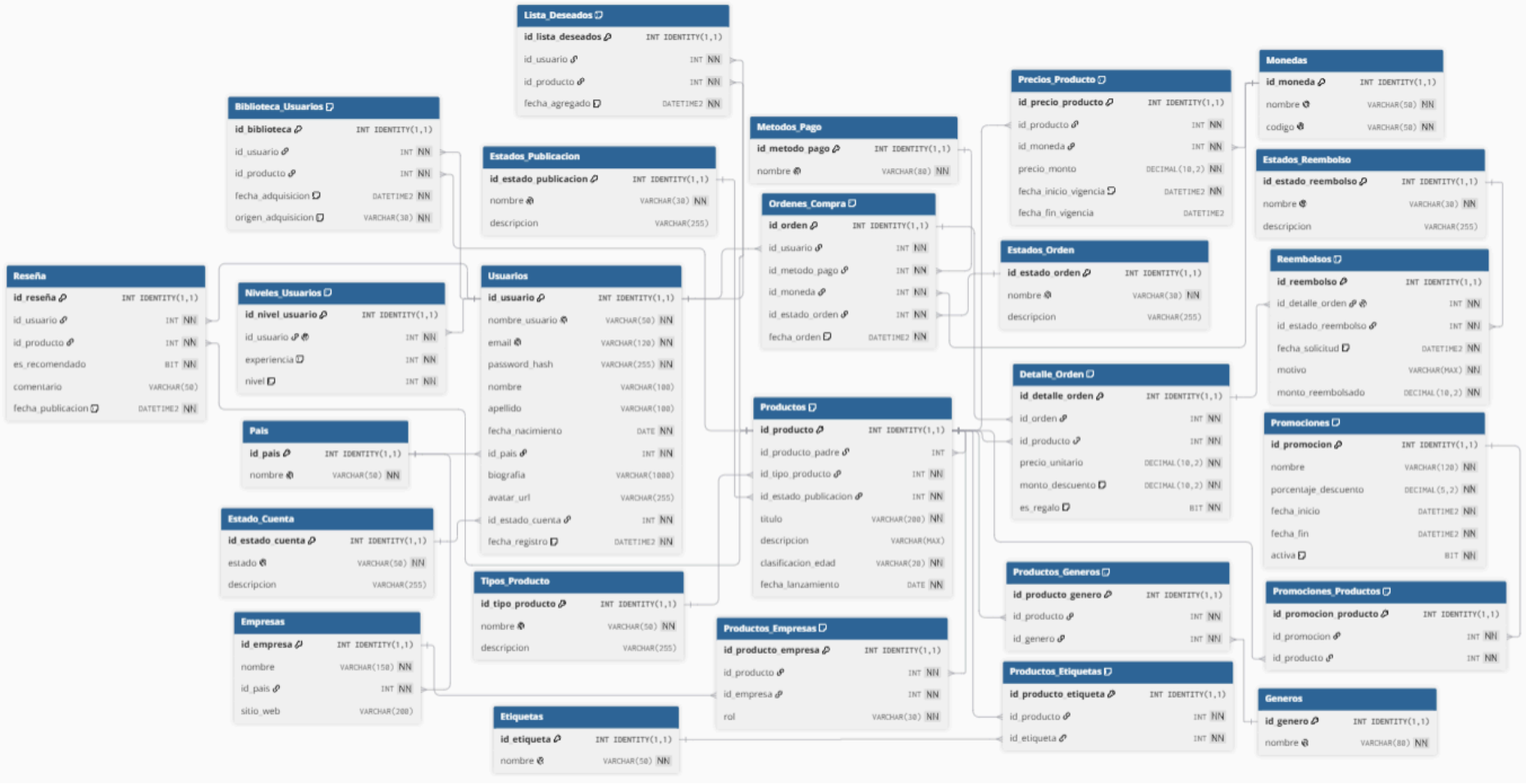
Cuando un usuario adquiere uno o más videojuegos, se genera una compra en la cual deben registrarse la fecha, el usuario, los videojuegos adquiridos, el precio abonado por cada uno y el medio de pago utilizado. Una compra puede contener varios videojuegos.

Además, los usuarios pueden agregar videojuegos a una lista de deseados (wishlist) para realizar un seguimiento de aquellos títulos que desean adquirir en el futuro. Un videojuego puede encontrarse en las listas de deseados de múltiples usuarios.

La plataforma permite realizar promociones y descuentos sobre determinados videojuegos durante períodos específicos. Para cada promoción se deberá registrar el porcentaje de descuento aplicado, la fecha y hora de inicio y finalización, y los videojuegos incluidos.
Un videojuego puede participar de diferentes promociones a lo largo del tiempo.

Los usuarios también pueden calificar los videojuegos que poseen mediante una reseña, indicando si recomiendan o no el videojuego y pudiendo agregar un comentario.
Estas reseñas deben quedar asociadas al usuario y al videojuego correspondiente.

2.-DIAGRAMA E-R:



3.-CONSULTAS

==========

421409 Lopez Repoma Kevin Andrei Jesus:

Crear una vista denominada vista_resumen_compras que muestre el número de orden,
la fecha de la transacción, el nombre de usuario del comprador, el título del videojuego adquirido,
la forma de pago utilizada y el importe final cobrado (precio unitario menos el descuento).
A partir de la vista creada, realizar una consulta que liste todas las transacciones registradas durante el año en curso,
ordenadas cronológicamente desde la más reciente a la más antigua.

```
-- Creación de la Vista
    CREATE VIEW vista_resumen_compras
    AS
    SELECT
        [o].[id_orden] AS Nro_Orden,
        [o].[fecha_orden] AS Fecha_Compra,
        [u].[nombre_usuario] AS Usuario,
        [p].[titulo] AS Videojuego,
        [mp].[nombre] AS Metodo_Pago,
        ([d].[precio_unitario] - [d].[monto_descuento]) AS Importe_Cobrado
    FROM Ordenes_Compra [o]
    JOIN Usuarios [u] ON [o].[id_usuario] = [u].[id_usuario]
    JOIN Metodos_Pago [mp] ON [o].[id_metodo_pago] = [mp].[id_metodo_pago]
    JOIN Detalle_Orden [d] ON [o].[id_orden] = [d].[id_orden]
    JOIN Productos [p] ON [d].[id_producto] = [p].[id_producto];
```

```
    GO
-- Consulta sobre la Vista creada
    SELECT
        Nro_Orden,
        Fecha_Compra,
        Usuario,
        Videojuego,
        Metodo_Pago,
        Importe_Cobrado
    FROM vista_resumen_compras
    WHERE YEAR(Fecha_Compra) = YEAR(GETDATE())
    ORDER BY Fecha_Compra DESC;
    GO
```

431713 Martinez Jalil Rodrigo Elias

```
/* Mostrar, para un año ingresado por el usuario,
la cantidad de órdenes, juegos vendidos, monto bruto,
descuentos e ingreso neto de cada mes, considerando solo órdenes completadas.
Sirve al gerente para ver la evolución de las ventas.   */

    CREATE OR ALTER FUNCTION fn_Ingresos_Mensuales (@anio INT)
    RETURNS TABLE
    AS
    RETURN
    (
    SELECT MONTH(oc.fecha_orden) AS mes,
        COUNT(DISTINCT oc.id_orden) AS cantidad_ordenes,
        COUNT(d.id_detalle_orden) AS juegos_vendidos,
        SUM(d.precio_unitario) AS monto_bruto,
```

```
        SUM(d.monto_descuento) AS descuentos,
        SUM(d.precio_unitario - d.monto_descuento) AS ingreso_neto
    FROM Ordenes_Compra oc
    JOIN Detalle_Orden d  ON d.id_orden = oc.id_orden
    JOIN Estados_Orden eo ON eo.id_estado_orden = oc.id_estado_orden
    WHERE eo.nombre = 'Completada'
    AND YEAR(oc.fecha_orden) = @anio
    GROUP BY MONTH(oc.fecha_orden)
    );
    GO

-- Consultas sobre la función
-- Todo el reporte de 2026.
    SELECT * FROM fn_Ingresos_Mensuales(2026) ORDER BY mes;

-- Solo los meses cuyo ingreso neto sea mayor a 100.
    SELECT * FROM fn_Ingresos_Mensuales(2026) WHERE ingreso_neto > 100;

--430197 Ruiz Gerónimo
--Mostrar los videojuegos que tengan al menos dos reseñas, indicando la cantidad
--total de reseñas, las positivas, las negativas y el porcentaje de recomendación.
--Agregar la cantidad de ventas completadas y los usuarios que tienen cada juego
--en su lista de deseados. Ordenar por mayor porcentaje de recomendación y ventas.

    SELECT
    p.titulo AS Videojuego,
    COUNT(*) AS Total_Resenas,
    SUM(CASE WHEN r.es_recomendado = 1 THEN 1 ELSE 0 END) AS Resenas_Positivas,
    SUM(CASE WHEN r.es_recomendado = 0 THEN 1 ELSE 0 END) AS Resenas_Negativas,
```

```
CAST(
SUM(CASE WHEN r.es_recomendado = 1 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)
AS DECIMAL(5,2)
) AS Porcentaje_Recomendacion,
(
SELECT COUNT(*)
FROM Detalle_Orden d
JOIN Ordenes_Compra oc ON d.id_orden = oc.id_orden
JOIN Estados_Orden eo ON oc.id_estado_orden = eo.id_estado_orden
WHERE d.id_producto = p.id_producto
        AND eo.nombre = 'Completada'
) AS Ventas_Completadas,
(
SELECT COUNT(DISTINCT ld.id_usuario)
FROM Lista_Deseados ld
WHERE ld.id_producto = p.id_producto
) AS Usuarios_Interesados
FROM Productos p
JOIN [Resena] r ON p.id_producto = r.id_producto
GROUP BY p.id_producto, p.titulo
HAVING COUNT(*) >= 2
ORDER BY Porcentaje_Recomendacion DESC, Ventas_Completadas DESC;
GO
```

/*109917 Minelli Santiago
4. Mostrar los desarrolladores cuyas ventas completadas de videojuegos superen al promedio de unidades vendidas
por desarrollador durante el año 2026.

Es necesario mostrar el nombre del desarrollador, la cantidad de títulos vendidos, el total de unidades vendidas y el importe total recaudado.

```
SELECT
    e.nombre 'Desarrollador',
    COUNT(DISTINCT p.id_producto) 'Titulos Vendidos',
    COUNT(do.id_detalle_orden) 'Unidades Vendidas',
    SUM(do.precio_unitario - do.monto_descuento) 'Importe Total'
FROM Empresas e
JOIN Productos_Empresas pe
    ON e.id_empresa = pe.id_empresa
JOIN Productos p
    ON pe.id_producto = P.id_producto
JOIN Detalle_Orden do
    ON p.id_producto = do.id_producto
JOIN Ordenes_Compra oc
    ON do.id_orden = oc.id_orden
JOIN Estados_Orden eo
    ON oc.id_estado_orden = eo.id_estado_orden
WHERE pe.rol = 'Desarrollador'
AND eo.nombre = 'Completada'
AND oc.fecha_orden >= '2026-01-01'
AND oc.fecha_orden < '2027-01-01'
GROUP BY e.id_empresa, e.nombre
HAVING COUNT(do.id_detalle_orden) >
(
SELECT AVG(CAST(ventas_completadas.Unidades AS DECIMAL(10,2)))
FROM
(
SELECT
```

```
        pe2.id_empresa,
        COUNT(do2.id_detalle_orden) 'Unidades'
    FROM Productos_Empresas pe2
    JOIN Detalle_Orden do2
        ON pe2.id_producto = do2.id_producto
    JOIN Ordenes_Compra oc2
        ON do2.id_orden = oc2.id_orden
    JOIN Estados_Orden eo2
        ON oc2.id_estado_orden = eo2.id_estado_orden
    WHERE pe2.rol = 'Desarrollador'
    AND eo2.nombre = 'Completada'
    AND oc2.fecha_orden >= '2026-01-01'
    AND oc2.fecha_orden < '2027-01-01'
    GROUP BY pe2.id_empresa
    ) AS Ventas_Completadas
    )
    ORDER BY 'Unidades Vendidas' DESC;
    GO


/*421654 - Gaiardo Gauna, Giorgio
5. "Mostrar los videojuegos base que nunca fueron vendidos en una orden completada.
Indicar el título, el género, el año de lanzamiento, la clasificación de edad y el precio vigente en
dólares. Ordenar por precio de mayor a menor y, en caso de empate, por título."
*/
    SELECT
        p.titulo                AS Videojuego,
        g.nombre                AS Genero,
        YEAR(p.fecha_lanzamiento)   AS Anio_Lanzamiento,
        p.clasificacion_edad    AS Clasificacion,
```

```
        pp.precio_monto          AS Precio_USD
FROM Productos p
JOIN Tipos_Producto tp      ON tp.id_tipo_producto = p.id_tipo_producto
JOIN Productos_Generos pg   ON pg.id_producto = p.id_producto
JOIN Generos g              ON g.id_genero = pg.id_genero
JOIN Precios_Producto pp    ON pp.id_producto = p.id_producto
JOIN Monedas m              ON m.id_moneda = pp.id_moneda
WHERE tp.nombre = 'Juego Base'
AND m.codigo = 'USD'
AND pp.fecha_fin_vigencia IS NULL
AND NOT EXISTS (
SELECT 1
FROM Detalle_Orden d
JOIN Ordenes_Compra oc ON oc.id_orden = d.id_orden
JOIN Estados_Orden eo  ON eo.id_estado_orden = oc.id_estado_orden
WHERE d.id_producto = p.id_producto
AND eo.nombre = 'Completada'
)
ORDER BY pp.precio_monto DESC, p.titulo;
GO
```

/*6. "Crear una vista denominada vista_gasto_usuarios que muestre, para cada usuario, su nombre de usuario, país, nivel, cantidad de órdenes completadas, cantidad de juegos comprados y el gasto total (precio unitario menos descuento). Incluir también a los usuarios sin compras, con valores en cero. A partir de la vista, listar los usuarios cuyo gasto total supere el gasto promedio de todos los usuarios, indicando el porcentaje que representa su gasto sobre el total recaudado y el gasto promedio por juego comprado.
Ordenar por gasto total de mayor a menor."
*/

```
CREATE VIEW vista_gasto_usuarios
AS
SELECT
        u.id_usuario                        AS Id_Usuario,
        u.nombre_usuario                    AS Usuario,
        pa.nombre                           AS Pais,
        nu.nivel                            AS Nivel,
        COUNT(DISTINCT oc.id_orden)         AS Ordenes_Completadas,
        COUNT(d.id_detalle_orden)           AS Juegos_Comprados,
        ISNULL(SUM(d.precio_unitario - d.monto_descuento), 0) AS Gasto_Total
FROM Usuarios u
JOIN Pais pa            ON pa.id_pais = u.id_pais
JOIN Niveles_Usuarios nu    ON nu.id_usuario = u.id_usuario
LEFT JOIN Ordenes_Compra oc ON oc.id_usuario = u.id_usuario
            AND oc.id_estado_orden = (SELECT id_estado_orden
                                      FROM Estados_Orden
                                      WHERE nombre = 'Completada')
LEFT JOIN Detalle_Orden d   ON d.id_orden = oc.id_orden
GROUP BY u.id_usuario, u.nombre_usuario, pa.nombre, nu.nivel;
GO

SELECT
        Usuario,
        Pais,
        Nivel,
        Ordenes_Completadas,
        Juegos_Comprados,
        Gasto_Total,
CAST(Gasto_Total * 100.0 / (SELECT SUM(Gasto_Total) FROM vista_gasto_usuarios)
```

```
    AS DECIMAL(5,2))                    AS Porcentaje_Del_Total,
        CAST(Gasto_Total / Juegos_Comprados AS DECIMAL(10,2)) AS
Gasto_Promedio_Por_Juego
    FROM vista_gasto_usuarios
    WHERE Gasto_Total > (SELECT AVG(Gasto_Total) FROM vista_gasto_usuarios)
    ORDER BY Gasto_Total DESC;
    GO

/*
Fullana Enzo 430253
Regla de negocio a la que responde:
Permite identificar usuarios inactivos, es decir, aquellos que no realizaron
una compra completada durante una determinada cantidad de días.
Esta información puede utilizarse para realizar campañas de recuperación de clientes.
Mostrando el id, nombre,
la fecha de su ultima compra y los dias que lleva inactivo.
*/
    CREATE FUNCTION dbo.fn_DiasDesdeUltimaCompra
    (
    @fechaUltimaCompra DATE
    )
    RETURNS INT
    AS
    BEGIN
        IF @fechaUltimaCompra IS NULL
    RETURN NULL;

    RETURN DATEDIFF(DAY, @fechaUltimaCompra, GETDATE());
    END;
```

```
GO

CREATE PROCEDURE dbo.sp_UsuariosInactivos
@diasInactividad INT
AS
BEGIN
	SELECT
		u.id_usuario,
		u.nombre_usuario,
		MAX(oc.fecha_orden) AS ultima_compra,
		dbo.fn_DiasDesdeUltimaCompra(MAX(oc.fecha_orden)) AS dias_inactivo
	FROM Usuarios u LEFT JOIN Ordenes_Compra oc ON u.id_usuario = oc.id_usuario
	LEFT JOIN Estados_Orden eo ON oc.id_estado_orden = eo.id_estado_orden
	WHERE eo.nombre = 'Completada' OR oc.id_orden IS NULL
	GROUP BY u.id_usuario, u.nombre_usuario
	HAVING MAX(oc.fecha_orden) IS NULL
	OR dbo.fn_DiasDesdeUltimaCompra(MAX(oc.fecha_orden)) >= @diasInactividad
	ORDER BY dias_inactivo DESC;
END;
GO
EXEC dbo.sp_UsuariosInactivos
@diasInactividad = 90;
GO
```